💰 AWS Cost Optimization — Category 11: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Intermediate → Advanced
> **Topics Covered:** Cost Explorer, Budgets, Cost Anomaly Detection, Savings Plans, Reserved Instances, Trusted Advisor, Compute Optimizer, Spot Strategy, S3 Storage Tiers, Data Transfer Costs

---

📋 Table of Contents

1. 11.1 Cost Explorer, Budgets, Cost Anomaly Detection
2. 11.2 Savings Plans vs Reserved Instances
3. 11.3 Trusted Advisor & Compute Optimizer
4. 11.4 Spot Strategy for Production Workloads
5. 11.5 S3 Intelligent-Tiering & Storage Cost Patterns
6. 11.6 Data Transfer Costs — The Hidden AWS Bill Killer

11.1 Cost Explorer, Budgets, Cost Anomaly Detection

🟢 What It Is in Simple Terms

AWS Cost Explorer visualizes and analyzes your AWS spending historically and forecasts future costs. AWS Budgets sends alerts when spending exceeds or is forecast to exceed thresholds. Cost Anomaly Detection uses ML to automatically identify unusual spending patterns before you notice them on the bill.

🔍 Why This Matters Operationally

```
Common cloud cost failure modes:
├── A developer leaves a dev RDS instance running over the weekend → $500
surprise
├── A misconfigured NAT Gateway processes 10TB of data → $450 bill
├── An auto-scaling group scales to 500 instances due to a bug → $10,000
overnight
├── S3 Glacier retrieval job runs accidentally → $2,000 expedited
retrieval bill
└── EC2 instances run for months after project ends → orphaned resource
waste

Without Cost Explorer + Budgets + Anomaly Detection:
→ Discover cost problems on the monthly billing statement (days/weeks too
late)
```

```
With proactive cost monitoring:
→ Alert fires within hours of anomaly
→ Budget threshold hit at 80% → investigate immediately
→ Cost Explorer drilldown → find the specific resource causing the spike
```

🧩 Cost Explorer

```
Cost Explorer = interactive cost and usage visualization tool

Key capabilities:
├── Visualize costs over time (daily, monthly, quarterly)
├── Filter by: service, region, account, tag, instance type, usage type
├── Group by: any dimension (service + tag combination)
├── Forecast: predict next 12 months based on historical patterns
├── Rightsizing recommendations: spot over-provisioned EC2
└── Reserved Instance and Savings Plans utilization/coverage reports

Dimensions you can filter and group by:
├── Service:         EC2, RDS, S3, Lambda, CloudFront, etc.
├── Linked account:  specific account in an org
├── Region:          us-east-1, eu-west-1, etc.
├── Instance type:   m5.2xlarge, r5.4xlarge, etc.
├── Tag:             Team=payments, Environment=prod
├── Usage type:      DataTransfer-Out, BoxUsage, RDS:Multi-AZ
├── Purchase option: On-Demand, Reserved, Spot, Savings Plans
└── Operating system: Linux, Windows, RHEL

CLI queries:
# Total cost by service for last month
aws ce get-cost-and-usage \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity MONTHLY \
  --metrics BlendedCost \
  --group-by '[{"Type":"DIMENSION","Key":"SERVICE"}]'

# Cost by Team tag, broken down by service
aws ce get-cost-and-usage \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity MONTHLY \
  --metrics UnblendedCost \
  --group-by '[{"Type":"TAG","Key":"Team"},
{"Type":"DIMENSION","Key":"SERVICE"}]' \
  --filter '{
    "Tags": {
      "Key": "Environment",
      "Values": ["prod"]
    }
  }'

# Get cost forecast for next 3 months
```

```
aws ce get-cost-forecast \
  --time-period Start=2024-02-01,End=2024-05-01 \
  --metric BLENDED_COST \
  --granularity MONTHLY

# Identify specific EC2 usage types driving cost
aws ce get-cost-and-usage \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity DAILY \
  --metrics UnblendedCost \
  --filter '{"Dimensions":{"Key":"SERVICE","Values":["Amazon Elastic
Compute Cloud - Compute"]}}' \
  --group-by '[{"Type":"DIMENSION","Key":"USAGE_TYPE"}]'
```

```
Cost Explorer rightsizing recommendations:
├── Analyzes EC2 CPU + memory utilization over 14 days
├── Recommends: downsize to smaller instance OR change family
├── Shows: estimated monthly savings per recommendation
└── Limitations: requires CloudWatch detailed monitoring enabled
                 only looks backward 14 days (may miss seasonal patterns)

EC2 rightsizing recommendation example:
Current:     m5.4xlarge ($0.768/hr = $553/month)
Average CPU: 8% (severely underutilized)
Recommended: m5.large ($0.096/hr = $69/month)
Savings:     $484/month per instance
```

🧩 AWS Budgets

```
Budgets = proactive cost and usage alerts
           Four budget types:
           ├── Cost budget:    alert when $ spending exceeds threshold
           ├── Usage budget:   alert when usage (GB, hours) exceeds
threshold
           ├── RI utilization: alert when Reserved Instance utilization
drops
           └── RI coverage:    alert when % of usage NOT covered by RI
drops

Budget notification triggers:
├── ACTUAL:   spending has already exceeded threshold
└── FORECASTED: spending is predicted to exceed threshold before period
ends

Multi-threshold example (common production setup):
Budget: Team=payments, $100,000/month
├── Alert 1: FORECASTED > 100% → "will exceed budget this month"
├── Alert 2: ACTUAL > 80%     → "at $80K, investigate now"
```

```
├── Alert 3: ACTUAL > 100%    → "over budget — page team lead"
└── Alert 4: ACTUAL > 120%    → "critical overage — escalate"
```

```
# Create cost budget with multiple alerts
aws budgets create-budget \
  --account-id 123456789012 \
  --budget '{
    "BudgetName":   "payments-team-prod-monthly",
    "BudgetType":   "COST",
    "TimeUnit":     "MONTHLY",
    "BudgetLimit":  {"Amount": "100000", "Unit": "USD"},
    "CostFilters": {
      "TagKeyValue": ["user:Team$payments"],
      "TagKeyValue": ["user:Environment$prod"]
    },
    "CostTypes": {
      "IncludeTax":      true,
      "IncludeSupport":  false,
      "IncludeDiscount": true
    }
  }' \
  --notifications-with-subscribers '[
    {
      "Notification": {
        "NotificationType":   "FORECASTED",
        "ComparisonOperator": "GREATER_THAN",
        "Threshold":          100,
        "ThresholdType":      "PERCENTAGE"
      },
      "Subscribers": [{"SubscriptionType":"EMAIL","Address":"payments-
lead@company.com"}]
    },
    {
      "Notification": {
        "NotificationType":   "ACTUAL",
        "ComparisonOperator": "GREATER_THAN",
        "Threshold":          80,
        "ThresholdType":      "PERCENTAGE"
      },
      "Subscribers": [
        {"SubscriptionType":"EMAIL","Address":"payments-
team@company.com"},
        {"SubscriptionType":"SNS",  "Address":"arn:aws:sns:...:cost-
alerts"}
      ]
    }
  ]'

# Budget Actions — automatically act when threshold exceeded
# Example: stop EC2 instances in dev account when dev budget exceeded
aws budgets create-budget-action \
```

```
  --account-id 123456789012 \
  --budget-name "dev-account-budget" \
  --notification-type ACTUAL \
  --action-type STOP_EC2_INSTANCES \
  --action-threshold
'{"ActionThresholdValue":100,"ActionThresholdType":"PERCENTAGE"}' \
  --definition '{
    "IamActionDefinition": {
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonEC2FullAccess",
      "Roles": ["arn:aws:iam::123:role/budget-action-role"]
    }
  }' \
  --approval-model AUTOMATIC
# When dev account exceeds 100% of budget → automatically stop all EC2
instances
```

🧩 Cost Anomaly Detection

```
Cost Anomaly Detection = ML-powered automatic anomaly identification
                         No thresholds to set — learns your spending
pattern
                         Alerts when spending deviates significantly from
baseline

How it works:
├── Trains on your historical cost patterns (service, account, region,
tag)
├── Accounts for: day-of-week patterns, growth trends, seasonal variation
├── Detects: unusual spikes that deviate from expected pattern
└── Sends alerts with: root cause analysis (which service/region/usage
type)

Without Anomaly Detection:
- Set Budget at 110% of expected → fires for any growth
- Too many false positives → teams stop paying attention
- Miss actual anomalies buried in normal growth

With Anomaly Detection:
- ML baseline: Tuesday spend is normally $5K
- Actual Tuesday spend: $47K → anomaly detected
- Root cause: "us-east-1 EC2 BoxUsage spiked $42K above baseline"
- Alert fires within hours → investigation begins

Monitor types:
├── AWS service: monitor all usage of specific service (e.g., EC2)
├── Linked account: monitor spending in a specific account
├── Cost category: group accounts/services into categories, monitor
together
└── Tag: monitor spending on resources with specific tag (e.g.,
Team=payments)
```

```
# Create anomaly monitor for all services
aws ce create-anomaly-monitor \
  --anomaly-monitor '{
    "MonitorName": "all-services-monitor",
    "MonitorType": "DIMENSIONAL",
    "MonitorDimension": "SERVICE"
  }'

# Create anomaly monitor for specific tag
aws ce create-anomaly-monitor \
  --anomaly-monitor '{
    "MonitorName": "payments-team-monitor",
    "MonitorType": "CUSTOM",
    "MonitorSpecification": {
      "Tags": {"Key": "Team", "Values": ["payments"]}
    }
  }'

# Create alert subscription (alert when anomaly > $500 total impact)
aws ce create-anomaly-subscription \
  --anomaly-subscription '{
    "SubscriptionName":   "high-impact-anomalies",
    "MonitorArnList":     ["arn:aws:ce::123:anomalymonitor/abc123"],
    "Subscribers": [{
      "Address":          "arn:aws:sns:us-east-1:123:cost-alerts",
      "Type":             "SNS"
    }],
    "Threshold":          500,
    "Frequency":          "DAILY"
  }'
# Alert when any single anomaly has > $500 total impact
# DAILY frequency: receive at most one alert per day per anomaly

# Get detected anomalies
aws ce get-anomalies \
  --date-interval '{"StartDate":"2024-01-01","EndDate":"2024-02-01"}' \
  --monitors '["arn:aws:ce::123:anomalymonitor/abc123"]'
```

```
Anomaly alert content:
{
  "anomalyId": "...",
  "anomalyStartDate": "2024-01-15",
  "anomalyScore": {"maxScore": 0.95, "currentScore": 0.95},
  "impact": {
    "maxImpact": 42000,      ← max single-day extra spend
    "totalImpact": 42000,    ← total extra spend in anomaly period
    "totalActualSpend": 47000,
    "totalExpectedSpend": 5000
  },
```

```
    "rootCauses": [{
      "service": "Amazon Elastic Compute Cloud - Compute",
      "region":  "us-east-1",
      "usageType": "BoxUsage:m5.4xlarge",
      "linkedAccount": "123456789012"
    }]
  }
```

💬 Short Crisp Interview Answer

"Cost Explorer is the analytical layer — drill into historical costs by service, tag, region, account, and identify exactly which resource type is driving spend. Budgets are proactive alerts — set monthly thresholds per team or environment with both FORECASTED (will exceed) and ACTUAL (has exceeded) notifications, and Budget Actions can automatically stop EC2 instances when a dev account exceeds its budget. Cost Anomaly Detection uses ML to establish your spending baseline per service and alerts when actual spend deviates significantly from expected — no threshold to tune, and the alert includes root cause analysis identifying which service and usage type caused the spike. In production, you want all three layers: Budgets for hard limits, Anomaly Detection for unexpected spikes, and Cost Explorer for investigation."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Cost data delay	Cost data has 8-24 hour delay. Yesterday's costs may not appear until today
Tag activation required	Cost Explorer can't filter by tags until cost allocation tags are activated in Billing
Rightsizing = 14 days only	EC2 rightsizing uses 14 days of data — misses seasonal patterns
Anomaly Detection training period	Needs ~10 days of historical data before useful anomaly detection
Budget filters = AND logic	Multiple CostFilters use AND logic — filter Team=payments AND Environment=prod
Anomaly frequency	DAILY = one alert per anomaly per day. IMMEDIATE = alert as soon as detected (noisier)
BlendedCost	Combines the costs of resources across linked accounts and applies average pricing, useful for consolidated billing.
UnblendedCost	Shows the actual cost incurred by each account or resource, reflecting the true price paid.

11.2 Savings Plans vs Reserved Instances

🟢 What It Is in Simple Terms

AWS On-Demand pricing is the highest rate — pay-as-you-go with no commitment. Savings Plans and Reserved Instances are commitment-based discounts where you commit to using a minimum amount of compute for 1 or 3 years, in exchange for discounts of 30-72% off On-Demand. Understanding the difference between them is essential for managing cloud spend at scale.

🔍 The Core Trade-off

```
On-Demand:
├── Maximum flexibility — stop anytime, change instance type anytime
└── Maximum price — pay full rate

Committed discounts (Savings Plans / Reserved Instances):
├── Commit to 1 or 3 years of specific usage
└── Receive 30-72% discount in exchange

The commitment spectrum:
More flexible ←──────────────────────────────────→ Less flexible
More expensive                                      Less expensive

Compute SP    EC2 Instance SP    EC2 Standard RI    EC2 Convertible RI
(~66% off)      (~72% off)         (~72% off)          (~66% off)
Most flexible                                         Least flexible but
                                                      can be exchanged
```

🧩 Savings Plans — Three Types

```
Savings Plans = commitment to spend $X/hour on compute for 1 or 3 years
                AWS applies discount to any matching usage automatically

1. Compute Savings Plans (most flexible):
   ├── Discount: up to 66% vs On-Demand
   ├── Applies to: EC2 (ANY instance family, size, OS, region),
   │               Lambda ($/GB-second),
   │               Fargate (vCPU + memory hours)
   ├── Flexibility: change instance family, size, OS, region freely
   └── Best for: organizations that use diverse EC2 types, Lambda, Fargate

2. EC2 Instance Savings Plans (better discount, less flexible):
   ├── Discount: up to 72% vs On-Demand
   ├── Applies to: specific EC2 instance FAMILY in specific region
   │               (e.g., M family in us-east-1)
   ├── Flexibility: change size within family (m5.large → m5.8xlarge)
   │               change OS (Linux → Windows) within family
   └── Best for: stable EC2 workloads in specific regions
```

```
3. SageMaker Savings Plans:
   ├── Discount: up to 64%
   ├── Applies to: SageMaker instance usage
   └── Best for: ML teams with consistent SageMaker usage

Savings Plans purchasing:
# All purchasing done via AWS Cost Explorer console or CLI
# Not via EC2 console — that's for Reserved Instances

aws savingsplans purchase-savings-plan \
  --savings-plan-type COMPUTE \
  --payment-option NO_UPFRONT \
  --duration-seconds 31536000 \
  --hourly-commitment 10.00
# Committing to spend $10/hour on compute for 1 year
# At $10/hr × 8760 hrs = $87,600/year commitment
```

🧩 Reserved Instances — Types and Options

```
Reserved Instances (RIs) = reserve specific EC2 capacity + get discount
                            Applied to specific instance type + region

RI scope:
├── Regional RI: discount applies to any instance in the region
│   (can be used across AZs within region)
└── Zonal RI:    reserves CAPACITY in specific AZ + discount
    Use zonal when: capacity reservation matters (ensures instance
launches)

RI types:
├── Standard RI:
│   ├── Discount: up to 72% vs On-Demand
│   ├── Can be modified: size (within family), AZ, scope
│   ├── Cannot be exchanged: stuck with instance family + OS
│   └── Can be sold on RI Marketplace if no longer needed
│
└── Convertible RI:
    ├── Discount: up to 66% vs On-Demand
    ├── Can be EXCHANGED: change instance family, OS, tenancy
    │   (must exchange for equal or greater value)
    └── Cannot be sold on RI Marketplace

RI payment options:
├── All Upfront (AU):   pay full amount now → deepest discount
├── Partial Upfront (PU): pay part now, rest monthly → medium discount
└── No Upfront (NU):    pay nothing now, higher monthly → least discount

For 1 year vs 3 years:
├── 1-year:  lower commitment risk, smaller discount (~40% vs OD)
```

```
└── 3-year:  highest commitment risk, deepest discount (~60-72% vs OD)

RIs for services OTHER than EC2:
├── RDS:         DB instance class + engine + Multi-AZ
├── ElastiCache: cache node type
├── Redshift:    node type
├── OpenSearch:  instance type
└── DynamoDB:    reserved capacity (RCU/WCU) — different model
```

🧩 Savings Plans vs Reserved Instances — Decision Framework

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Scenario             │ Recommendation       │ Reason               │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Stable EC2, known    │ EC2 Instance SP or   │ Maximum discount for │
│ instance family      │ Standard RI          │ predictable workload │
│                      │                      │                      │
│ Mixed EC2 types,     │ Compute SP           │ Flexibility to change│
│ changing over time   │                      │ families/sizes       │
│                      │                      │                      │
│ Lambda workloads     │ Compute SP           │ Only SP covers Lambda│
│                      │                      │ (RIs don't apply)    │
│                      │                      │                      │
│ Fargate containers   │ Compute SP           │ Only SP covers       │
│                      │                      │ Fargate              │
│                      │                      │                      │
│ RDS databases        │ RDS Reserved         │ SPs don't cover RDS  │
│                      │ Instances            │                      │
│                      │                      │                      │
│ Uncertain growth,    │ Compute SP           │ Can absorb new usage │
│ will scale up        │ No Upfront           │ without over-buying  │
│                      │                      │                      │
│ Need capacity        │ Zonal RI             │ Only Zonal RI        │
│ reservation          │                      │ reserves capacity    │
└──────────────────────┴──────────────────────┴──────────────────────┘

The recommendation cascade:
1. Identify stable baseline usage (what always runs 24/7)
2. Buy Savings Plans / RIs to cover that baseline
3. Let spikes above baseline use On-Demand
4. Use Spot Instances for flexible workloads on top

Commitment sizing strategy:
Don't commit to 100% of usage — you'll waste money if you scale down.
Commit to 70-80% of your MINIMUM expected usage.
├── 70-80% covered by SP/RI at discounted rate
├── 20-30% covered by On-Demand (flex for spikes + safety margin)
└── Additional capacity: Spot Instances for non-critical work
```

🧩 Utilization and Coverage Reports

```
RI/SP Utilization = % of your commitment you actually used
SP/RI Coverage   = % of your total usage covered by SP/RI

Target metrics:
├── Utilization: > 90% (if lower → over-bought, wasting commitment)
└── Coverage:    > 80% (if lower → opportunity to buy more)

Utilization = Commitment Used / Total Commitment × 100

Example:
Committed: $10/hr Compute SP
Actually used: $8/hr covered by SP
Utilization: 80% (underutilizing $2/hr → wasted commitment)
→ Action: reduce next commitment or ensure utilization increases

Coverage = SP/RI-covered usage / Total usage × 100

Example:
Total EC2 cost: $50,000
SP/RI covered: $35,000
On-Demand:     $15,000
Coverage:      70%
→ Action: buy more SP to cover more On-Demand usage

aws ce get-savings-plans-utilization \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity MONTHLY

aws ce get-savings-plans-coverage \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity MONTHLY \
  --group-by '[{"Type":"DIMENSION","Key":"SERVICE"}]'
```

💬 Short Crisp Interview Answer

"Savings Plans and Reserved Instances both provide discounts of 30-72% in exchange for 1 or 3-year commitments. Savings Plans are more flexible: Compute Savings Plans apply to ANY EC2 instance regardless of family, size, region, plus Lambda and Fargate — ideal for dynamic environments. EC2 Instance Savings Plans commit to a specific instance family in a region but allow size changes within that family. Reserved Instances are more rigid — Standard RIs are cheapest but can't change family, Convertible RIs can be exchanged but at lower discount. The key rule: RDS, ElastiCache, and Redshift only support Reserved Instances, not Savings Plans. Commit to 70-80% of your minimum baseline usage, not peak — over-committing wastes money. Track utilization (are you using what you committed?) and coverage (what % of usage is on-demand vs committed)."

⚠️ Tricky Edge Cases / Gotchas

Gotcha	Detail
Savings Plans don't cover RDS	RDS requires its own Reserved Instances — no Savings Plan covers it
Zonal vs Regional RI capacity	Only Zonal RI guarantees capacity — Regional RI doesn't reserve physical capacity
SP/RI applied automatically	AWS applies SPs/RIs to matching usage automatically — no manual assignment needed
Convertible RI = no Marketplace	Can be exchanged but cannot be sold on RI Marketplace like Standard RIs
Over-commitment is expensive	Unused SP/RI hours are wasted — commit to minimum baseline usage, not peak
1-year vs 3-year break-even	3-year saves more but risky — only commit 3 years for very stable, long-term workloads

---

11.3 Trusted Advisor & Compute Optimizer

🟢 What It Is in Simple Terms

Trusted Advisor automatically audits your AWS account across five pillars and gives specific actionable recommendations. Compute Optimizer uses ML to analyze actual utilization metrics and recommends optimal instance types and sizes for EC2, Lambda, ECS, and EBS. Together they're your automated cost and efficiency review team.

🧩 AWS Trusted Advisor

```
Trusted Advisor checks five pillars:
├── Cost Optimization
├── Performance
├── Security
├── Fault Tolerance
└── Service Limits (Service Quotas)

Check tiers by support plan:
├── Basic/Developer: ~6 core checks (security groups, S3 permissions, MFA
on root)
├── Business/Enterprise: ALL checks (200+ checks across all 5 pillars)
└── Enterprise: programmatic access to all checks + AWS Health API

Key COST checks (Business+ support):
├── Low-utilization EC2 instances:
│   CPU < 10% for 14 days + network < 5 MB/day
```

```
│   → Recommendation: downsize or terminate
│
├── Idle RDS instances:
│   0 connections for 14 days
│   → Recommendation: stop or delete
│
├── Idle load balancers:
│   < 100 requests/day for 14 days
│   → Recommendation: delete
│
├── Underutilized EBS volumes:
│   Volume not attached to running instance
│   → Recommendation: snapshot and delete
│
├── Unassociated Elastic IPs:
│   Elastic IP not associated with running instance
│   → Cost: $0.005/hr per unassociated EIP
│   → Recommendation: release
│
└── Reserved Instance optimization:
    Recommends purchase of RIs based on usage patterns

Key SECURITY checks (some available on all plans):
├── Security groups: open to 0.0.0.0/0 on sensitive ports
├── S3 bucket public access
├── IAM root account MFA not enabled
├── IAM access key rotation (keys > 90 days old)
├── CloudTrail not enabled
└── EBS public snapshots

Key FAULT TOLERANCE checks:
├── EC2 in single AZ (no multi-AZ deployment)
├── RDS Multi-AZ not enabled
├── EBS not in snapshot schedule
├── Route53 failover not configured
└── Auto Scaling groups with < 2 instances
```

```
# List all Trusted Advisor checks
aws support describe-trusted-advisor-checks --language en

# Get results for a specific check
# First get check ID from describe-trusted-advisor-checks
aws support describe-trusted-advisor-check-result \
  --check-id "Di4UjvhlS9" \
  --language en

# Refresh a check (force re-evaluation)
aws support refresh-trusted-advisor-check --check-id "Di4UjvhlS9"

# Get summary of all checks
aws support describe-trusted-advisor-checks-summary \
```

```
  --check-ids "Di4UjvhlS9" "Qch7DwouX1"

# Automate: check for unassociated Elastic IPs daily
# Schedule Lambda → describe-trusted-advisor-check-result → alert if
flagged resources
```

```
Trusted Advisor + EventBridge automation:
TA check status changes → EventBridge → Lambda → auto-remediation

Example:
Trusted Advisor: "EIP not associated" → EventBridge → Lambda:
  → Check if EIP truly unused (not being set up)
  → If unused > 7 days → release EIP automatically
  → Notify team with released EIP details

aws events put-rule \
  --name "trusted-advisor-alerts" \
  --event-pattern '{
    "source": ["aws.trustedadvisor"],
    "detail-type": ["Trusted Advisor Check Item Refresh Notification"],
    "detail": {
      "status": ["ERROR", "WARN"],
      "check-name": ["Low Utilization Amazon EC2 Instances"]
    }
  }'
```

🧩 AWS Compute Optimizer

```
Compute Optimizer = ML-driven right-sizing recommendations
                    Uses CloudWatch metrics (14 days default, 3 months
optional)
                    to recommend optimal compute configuration

Supported resources:
├── EC2 instances
├── EC2 Auto Scaling groups
├── EBS volumes
├── Lambda functions (memory configuration)
└── ECS services on Fargate (CPU + memory)

How it works:
1. Analyze CloudWatch metrics for your resources:
   EC2:     CPUUtilization, MemoryUtilization (if CW Agent installed),
            NetworkIn/Out, DiskReadBytes, DiskWriteBytes
   Lambda:  Duration, MaxMemoryUsed, Invocations
   EBS:     VolumeReadOps, VolumeWriteOps, VolumeThroughput
2. ML model determines utilization patterns
3. Recommends: different instance type + estimated savings
```

```
Findings categories per resource:
├── Over-provisioned: using < 40% of resources → downsizing recommended
├── Under-provisioned: consistently hitting limits → upsizing recommended
├── Optimized:         current type is appropriate → no change needed
└── Not enough data:   insufficient CloudWatch metrics → enable monitoring

EC2 recommendation example:
Current:     m5.4xlarge ($0.768/hr)  CPU avg: 12%  Memory avg: 15%
Recommended: m5.xlarge  ($0.192/hr)  Handles same workload
Savings:     75% cost reduction = $416/month per instance
Risk:        LOW (95th percentile CPU still < 60% on m5.xlarge)
```

```
# Get EC2 instance recommendations
aws compute-optimizer get-ec2-instance-recommendations \
  --instance-arns arn:aws:ec2:us-east-1:123:instance/i-0abc123

# Get recommendations for all instances in account
aws compute-optimizer get-ec2-instance-recommendations

# Get Lambda recommendations
aws compute-optimizer get-lambda-function-recommendations \
  --function-arns arn:aws:lambda:us-east-1:123:function:my-function

# Get EBS volume recommendations
aws compute-optimizer get-ebs-volume-recommendations \
  --volume-arns arn:aws:ec2:us-east-1:123:volume/vol-0abc123

# Export all recommendations to S3
aws compute-optimizer export-ec2-instance-recommendations \
  --s3-destination-config BucketName=my-optimizer-bucket,KeyPrefix=ec2/
```

```
Compute Optimizer recommendation detail:
{
  "instanceArn": "arn:aws:ec2:...:instance/i-0abc123",
  "currentInstanceType": "m5.4xlarge",
  "finding": "OVER_PROVISIONED",
  "utilizationMetrics": [
    {"name": "CPU", "statistic": "MAXIMUM", "value": 18.5},
    {"name": "MEMORY", "statistic": "MAXIMUM", "value": 22.1}
  ],
  "recommendationOptions": [
    {
      "instanceType":         "m5.large",
      "estimatedMonthlySavings": {"currency": "USD", "value": 405.50},
      "performanceRisk":      0.1,   ← 0=no risk, 1=high risk
      "rank": 1                     ← best recommendation
    },
    {
```

```
      "instanceType":         "m5.xlarge",
      "estimatedMonthlySavings": {"currency": "USD", "value": 350.20},
      "performanceRisk":      0.0,   ← zero risk
      "rank": 2
    }
  ]
}
```

🧩 Trusted Advisor vs Compute Optimizer

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Feature              │ Trusted Advisor      │ Compute Optimizer    │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Analysis type        │ Rule-based checks    │ ML-based analysis    │
│ Metrics used         │ Basic CW metrics     │ Deep CW metric analysis│
│ Scope                │ 5 pillars            │ Compute only         │
│ Recommendation detail│ Medium               │ Very detailed        │
│ Performance risk     │ Not shown            │ Quantified 0-1       │
│ Services covered     │ Broad (all AWS)      │ EC2, ASG, Lambda,    │
│                      │                      │ EBS, ECS Fargate     │
│ Cost                 │ Business+ support    │ Free (basic)         │
│ Memory metrics       │ No (basic CPU only)  │ Yes (with CW Agent)  │
└──────────────────────┴──────────────────────┴──────────────────────┘

Use both:
├── Trusted Advisor: broad sweep across all services and pillars
│   Catch: idle RDS, unattached EBS, open security groups, service limits
└── Compute Optimizer: deep compute right-sizing
    Catch: over-provisioned EC2, suboptimal Lambda memory, EBS gp2 → gp3
upgrade
```

💬 Short Crisp Interview Answer

"Trusted Advisor audits your account across cost, performance, security, fault tolerance, and service limits using rule-based checks — requires Business or Enterprise support for the full 200+ check library. It catches idle resources (RDS with 0 connections for 14 days, EC2 with < 10% CPU), security issues, and fault tolerance gaps. Compute Optimizer is a deeper ML-based analysis specifically for compute right-sizing — it analyzes 14 days of CloudWatch metrics for EC2, Lambda, EBS, and Fargate and recommends specific instance types with quantified performance risk (0-1 scale). A key gotcha: Compute Optimizer's memory recommendations require CloudWatch Agent installed on EC2 — without it, memory utilization is unknown and recommendations may be incomplete. Use Trusted Advisor for the broad sweep, Compute Optimizer for precise compute sizing."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Trusted Advisor = Business+ support	Most checks require paid support tier — free tier only gets ~6 core checks
Compute Optimizer memory	Memory-based recommendations need CloudWatch Agent — without it, may wrongly suggest downsizing
TA checks are periodic	Trusted Advisor refreshes checks on its own schedule — use refresh API to force update
Compute Optimizer opt-in	Must opt in per account or organization — not enabled by default
14-day lookback window	May miss seasonal patterns, weekend vs weekday patterns — use 3-month lookback when available
Low-utilization ≠ idle	10% average CPU might be fine for bursty workloads — always check P99 before resizing

---

11.4 Spot Strategy for Production Workloads

🟢 What It Is in Simple Terms

Spot Instances are spare EC2 capacity that AWS offers at 60-90% discounts off On-Demand price. The trade-off: AWS can reclaim Spot Instances with a 2-minute warning when capacity is needed. A naive Spot strategy makes production fragile. A sophisticated Spot strategy saves massive amounts while maintaining resilience.

🔍 The Core Challenge

```
Naive Spot usage (fragile):
├── Launch single Spot instance for web server
├── AWS reclaims it at 3 AM
├── Service is down until someone manually intervenes
└── Result: "Spot is too risky for production" — wasted savings
opportunity

Sophisticated Spot usage (resilient):
├── Use instance fleet across MANY instance types and AZs
├── Spot interruption triggers Auto Scaling group to launch replacement
├── New instance running before users notice the interruption
└── Result: 70–90% cost savings with minimal availability impact

The key insight:
Individual Spot instances are unreliable.
```

```
A FLEET of Spot instances across many pools is reliable.
Probability of ALL pools being interrupted simultaneously ≈ 0.
```

🧩 Spot Pricing and Interruption Rates

```
Spot pricing:
├── Set by AWS based on spare capacity supply and demand
├── Prices change (but no longer fluctuate wildly — more stable than
before)
├── Typical discount: 60-90% vs On-Demand
└── Example: m5.xlarge On-Demand = $0.192/hr, Spot = $0.057/hr (70% off)

Spot Interruption:
├── AWS sends 2-minute interruption notice before termination
├── Notice via: EC2 instance metadata, EventBridge, CloudWatch Events
├── Average interruption rate: ~5% of Spot instances per month
│   (most instances run for hours without interruption)
└── Interruption frequency varies by: instance type, AZ, time of day

Spot Instance Advisor (AWS tool):
├── Shows interruption frequency per instance type per AZ
├── Categories: < 5%, 5-10%, 10-15%, 15-20%, > 20%
└── Strategy: use instance types with < 5% interruption frequency

Checking interruption notice (in Lambda/EC2):
# Poll IMDS for interruption notice
import urllib.request

def check_spot_interruption():
    try:
        response = urllib.request.urlopen(
            'http://169.254.169.254/latest/meta-data/spot/termination-
time',
            timeout=1
        )
        # If this URL returns data → interruption scheduled
        termination_time = response.read()
        print(f"SPOT INTERRUPTION SCHEDULED: {termination_time}")
        # Trigger graceful shutdown: drain connections, checkpoint state,
etc.
        return True
    except:
        return False  # No interruption scheduled
```

🧩 Spot Strategies in Auto Scaling Groups

```
Launch Template Spot configuration:
The modern approach: define MULTIPLE instance types in one launch
template.
Never use a single instance type for Spot in production.

# Launch template with multiple instance types
aws ec2 create-launch-template \
  --launch-template-name prod-web-lt \
  --launch-template-data '{
    "ImageId":      "ami-0abc123",
    "KeyName":      "prod-key",
    "SecurityGroupIds": ["sg-abc123"],
    "IamInstanceProfile": {"Arn": "arn:aws:iam::123:instance-profile/web-
role"},
    "UserData": "base64-encoded-startup-script"
  }'

# Auto Scaling Group with mixed instances policy
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name prod-web-asg \
  --min-size 10 \
  --max-size 100 \
  --desired-capacity 20 \
  --vpc-zone-identifier "subnet-az1,subnet-az2,subnet-az3" \
  --mixed-instances-policy '{
    "LaunchTemplate": {
      "LaunchTemplateSpecification": {
        "LaunchTemplateName": "prod-web-lt",
        "Version": "$Latest"
      },
      "Overrides": [
        {"InstanceType": "m5.xlarge"},
        {"InstanceType": "m5a.xlarge"},
        {"InstanceType": "m4.xlarge"},
        {"InstanceType": "m5n.xlarge"},
        {"InstanceType": "m5d.xlarge"},
        {"InstanceType": "r5.large"},
        {"InstanceType": "r5a.large"}
      ]
    },
    "InstancesDistribution": {
      "OnDemandBaseCapacity":                2,
      "OnDemandPercentageAboveBaseCapacity": 20,
      "SpotAllocationStrategy":             "price-capacity-optimized"
    }
  }'
```

```
Key ASG mixed instances parameters:

OnDemandBaseCapacity:                2
→ Always keep 2 On-Demand instances (guaranteed availability baseline)
```

```
→ These never interrupted — safety net

OnDemandPercentageAboveBaseCapacity: 20
→ Above the 2-instance baseline: 20% On-Demand, 80% Spot
→ At 20 total instances: 2 OD base + 18 flexible (3-4 OD + 14-15 Spot)

SpotAllocationStrategy options:
├── price-capacity-optimized (RECOMMENDED):
│   AWS picks pools with lowest price AND available capacity
│   Reduces interruption probability + gets good pricing
│
├── capacity-optimized:
│   AWS picks pools with most available capacity
│   Minimizes interruptions — good for fault-sensitive workloads
│
├── lowest-price:
│   AWS picks cheapest Spot pools
│   Higher interruption risk — use only for batch/non-critical workloads
│
└── diversified:
    Distributes across all specified instance types evenly
    Reduces risk of entire fleet being same pool (all interrupted
together)
```

🧩 Spot for Specific Workload Patterns

```
Pattern 1: Stateless web tier (most common Spot use case)
├── Web servers: stateless HTTP, any request can go to any instance
├── Load balancer removes interrupted instance from rotation
├── ASG replaces within 1-3 minutes
└── Users: 0.1% chance of seeing a connection reset during interruption

Architecture:
ALB → ASG (2 OD base + 80% Spot across 7 instance types, 3 AZs)
With 20 instances across 21 Spot pools (7 types × 3 AZs):
→ Probability of disruption at any moment ≈ 5%/pool × 1/21 pools ≈ 0.24%

Pattern 2: Batch processing (ideal for Spot)
├── Processing is interruptible and restartable
├── Checkpoint progress to S3 → resume from checkpoint on new instance
├── AWS Batch natively supports Spot + checkpointing
└── Cost savings: 80-90% vs On-Demand for batch jobs

Pattern 3: Data processing pipelines (Spark/EMR)
├── EMR natively supports Spot for task nodes
├── Core nodes (hold HDFS data): On-Demand
├── Task nodes (compute only): Spot (cheap, interruptible)
└── Spot interruption: task retried on another node automatically

# EMR with Spot task nodes
```

```
aws emr create-cluster \
  --instance-groups '[
    {
      "InstanceGroupType": "MASTER",
      "InstanceType": "m5.xlarge",
      "InstanceCount": 1,
      "Market": "ON_DEMAND"
    },
    {
      "InstanceGroupType": "CORE",
      "InstanceType": "m5.xlarge",
      "InstanceCount": 2,
      "Market": "ON_DEMAND"       ← Core on OD: holds data
    },
    {
      "InstanceGroupType": "TASK",
      "InstanceType": "m5.xlarge",
      "InstanceCount": 10,
      "Market": "SPOT",           ← Task on Spot: compute only
      "BidPrice": "0.10"          ← max bid = $0.10/hr
    }
  ]'

Pattern 4: CI/CD build agents
├── Jenkins / GitHub Actions self-hosted runners on Spot
├── Build triggered → Spot runner starts → build completes → terminates
├── If interrupted: build retried on new instance (most CI systems support
this)
└── Savings: 70-90% on build infrastructure
```

🧩 Spot Interruption Handling

```
EventBridge Spot interruption automation:
# When Spot interruption notice fires → graceful shutdown

aws events put-rule \
  --name "spot-interruption-handler" \
  --event-pattern '{
    "source": ["aws.ec2"],
    "detail-type": ["EC2 Spot Instance Interruption Warning"],
    "detail": {
      "instance-action": ["terminate"]
    }
  }'

# Lambda handler for graceful shutdown
def handle_spot_interruption(event, context):
    instance_id = event['detail']['instance-id']

    # 1. Deregister from load balancer target group
```

```
    elbv2.deregister_targets(
        TargetGroupArn=TARGET_GROUP_ARN,
        Targets=[{'Id': instance_id}]
    )

    # 2. Drain in-flight requests (wait up to 90 seconds)
    time.sleep(90)

    # 3. Checkpoint application state to S3/DynamoDB

    # 4. Notify Auto Scaling group to launch replacement
    autoscaling.terminate_instance_in_auto_scaling_group(
        InstanceId=instance_id,
        ShouldDecrementDesiredCapacity=False  # replace, don't shrink
    )

    # 5. Alert ops team
    sns.publish(TopicArn=ALERTS_TOPIC, Message=f"Spot interrupted:
{instance_id}")

Spot best practices:
├── Never use Spot for databases (stateful, interruption = data risk)
├── Never use Spot for single-point-of-failure components
├── Always use at least 4+ instance types for diversity
├── Always spread across 3+ AZs
├── Use price-capacity-optimized allocation strategy
├── Set OnDemandBaseCapacity for minimum guaranteed capacity
└── Always have EventBridge rule for interruption handling
```

💬 Short Crisp Interview Answer

"Spot Instances offer 60-90% discounts but can be reclaimed with 2-minute notice. The resilience strategy is using a fleet approach: Auto Scaling Groups with mixed instances policy, 6-8 instance types, 3 AZs, and price-capacity-optimized allocation strategy — AWS picks pools with both good pricing AND available capacity. Set OnDemandBaseCapacity to 2-3 for a guaranteed floor, and 20% On-Demand above that for the flexible portion. An EventBridge rule on EC2 Spot Instance Interruption Warning triggers a Lambda that deregisters from the load balancer, drains in-flight requests for 90 seconds, and tells the ASG to replace without decrementing desired capacity. Best workloads: stateless web tier, batch processing, Spark/EMR task nodes, CI/CD runners. Never use Spot for stateful services or single-point-of-failure components."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
2 minutes is not a lot of time	2-minute warning: deregister, drain, checkpoint — design for quick graceful shutdown
Spot pools can all be reclaimed	If you use only 1 instance type in 1 AZ, entire fleet can be reclaimed simultaneously

Gotcha	Detail
price-capacity-optimized vs lowest-price	lowest-price often picks high-interruption pools — use price-capacity-optimized
Spot for databases = bad	Stateful services with Spot = data corruption risk on interruption
Hibernation alternative	Some instances support Spot hibernation — save RAM to EBS instead of terminating
Bid price deprecated	Modern Spot doesn't require bid prices — just specify max On-Demand price as implicit ceiling

11.5 S3 Intelligent-Tiering & Storage Cost Patterns

🟢 What It Is in Simple Terms

S3 has seven storage tiers with different cost/latency/durability trade-offs. Intelligent-Tiering automatically moves objects between tiers based on access patterns without any retrieval fees — it's the default choice for most data whose access patterns are unpredictable. Understanding the full tier spectrum lets you optimize storage costs by 60-80%.

🧩 S3 Storage Classes — Full Spectrum

```
S3 Storage Classes (from most expensive + fastest to cheapest + slowest):

┌──────────────────────────┬──────────────┬────────────┬──────────────
───┐
│ Storage Class            │ Storage Cost │ Retrieval  │ Minimum
Duration │
├──────────────────────────┼──────────────┼────────────┼──────────────
───┤
│ S3 Standard              │ $0.023/GB    │ Free       │ None
│
│ (frequently accessed)    │              │            │
│
├──────────────────────────┼──────────────┼────────────┼──────────────
───┤
│ S3 Intelligent-Tiering   │ $0.023/GB    │ Free*      │ None
│
│ (auto-tiering)           │ (frequent)   │ (no fees)  │ 30-day min for
│
│                          │ $0.0125/GB   │            │ infrequent
tier  │
│                          │ (infrequent) │            │
│
├──────────────────────────┼──────────────┼────────────┼──────────────
```

```
────┤
│ S3 Standard-IA              │ $0.0125/GB   │ $0.01/GB   │ 30 days
│
│ (infrequent access)         │              │            │
│
├────────────────────────────┼──────────────┼────────────┼────────────────
────┤
│ S3 One Zone-IA              │ $0.01/GB     │ $0.01/GB   │ 30 days
│
│ (single AZ, infreq access)  │              │            │ (one AZ only!)
│
├────────────────────────────┼──────────────┼────────────┼────────────────
────┤
│ S3 Glacier Instant          │ $0.004/GB    │ $0.03/GB   │ 90 days
│
│ (archive, ms retrieval)     │              │            │
│
├────────────────────────────┼──────────────┼────────────┼────────────────
────┤
│ S3 Glacier Flexible         │ $0.0036/GB   │ $0.01/GB   │ 90 days
│
│ (archive, min-hr retrieval) │              │ minutes-   │
│
│                             │              │ 12 hours   │
│
├────────────────────────────┼──────────────┼────────────┼────────────────
────┤
│ S3 Glacier Deep Archive     │ $0.00099/GB  │ $0.02/GB   │ 180 days
│
│ (coldest, 12-48hr retrieval)│              │ 12-48 hours│
│
└────────────────────────────┴──────────────┴────────────┴────────────────
────┘

Cost comparison for 1 TB stored 1 year, accessed occasionally:
Standard:          $23.55/month × 12 = $282.60/year
Standard-IA:       $12.80/month × 12 = $153.60/year + retrieval costs
Glacier Deep Archive: $1.01/month × 12 = $12.12/year + retrieval costs
```

🧩 S3 Intelligent-Tiering — Deep Dive

```
Intelligent-Tiering tiers (automatic movement between tiers):

Tier 1: Frequent Access Tier       (standard pricing — accessed data lives
here)
Tier 2: Infrequent Access Tier     (40% cheaper — data not accessed in 30
days)
Tier 3: Archive Instant Access Tier (68% cheaper — data not accessed in 90
days)
         [Optional — must enable]
```

```
Tier 4: Archive Access Tier        (95% cheaper — data not accessed in 180
days)
         [Optional — must enable]
Tier 5: Deep Archive Access Tier   (99% cheaper — data not accessed in 730
days)
         [Optional — must enable]

How Intelligent-Tiering works:
1. Upload object with INTELLIGENT_TIERING storage class
2. Object starts in Frequent Access Tier
3. If not accessed for 30 consecutive days → moved to Infrequent Access
Tier
4. If accessed → instantly moved back to Frequent Access Tier (no
retrieval fee!)
5. If not accessed for 90 days → Archive Instant Access (if enabled)
6. Access from any tier → free retrieval (no per-GB retrieval charge)

Cost: $0.00025/object/month monitoring fee (for objects > 128KB)
     + tiered storage rate based on which tier object is in

When NOT to use Intelligent-Tiering:
├── Objects < 128KB: monitoring fee makes it more expensive than Standard
└── Objects accessed ALWAYS within 30 days: Standard is simpler + cheaper
    (Intelligent-Tiering monitoring fee added without any benefit)

S3-IT configuration:
aws s3api put-object \
  --bucket my-bucket \
  --key data/report-2024-01.parquet \
  --body report.parquet \
  --storage-class INTELLIGENT_TIERING

# Enable optional archive tiers
aws s3api put-bucket-intelligent-tiering-configuration \
  --bucket my-bucket \
  --id "archive-config" \
  --intelligent-tiering-configuration '{
    "Id": "archive-config",
    "Status": "Enabled",
    "Tierings": [
      {"Days": 90,  "AccessTier": "ARCHIVE_INSTANT_ACCESS"},
      {"Days": 180, "AccessTier": "ARCHIVE_ACCESS"},
      {"Days": 730, "AccessTier": "DEEP_ARCHIVE_ACCESS"}
    ]
  }'
```

🧩 S3 Lifecycle Policies

```
Lifecycle Policy = rules that automatically transition/expire objects over
time
```

```
Use when: access pattern is KNOWN and predictable
          (Intelligent-Tiering = unknown access pattern)

Example: log archival lifecycle
- Days 0-30:   S3 Standard (actively analyzed)
- Days 30-90:  S3 Standard-IA (occasionally referenced)
- Days 90-365: S3 Glacier Instant Retrieval (rare reference)
- Days 365+:   S3 Glacier Deep Archive (compliance retention only)
- Days 2555:   DELETE (7-year retention complete)
```

```
aws s3api put-bucket-lifecycle-configuration \
  --bucket prod-logs \
  --lifecycle-configuration '{
    "Rules": [{
      "ID": "log-archival",
      "Status": "Enabled",
      "Filter": {"Prefix": "access-logs/"},
      "Transitions": [
        {"Days": 30,  "StorageClass": "STANDARD_IA"},
        {"Days": 90,  "StorageClass": "GLACIER_IR"},
        {"Days": 365, "StorageClass": "DEEP_ARCHIVE"}
      ],
      "Expiration": {"Days": 2555},
      "NoncurrentVersionTransitions": [
        {"NoncurrentDays": 30, "StorageClass": "STANDARD_IA"},
        {"NoncurrentDays": 90, "StorageClass": "GLACIER_IR"}
      ],
      "NoncurrentVersionExpiration": {"NoncurrentDays": 365}
    }]
  }'
```

```
Lifecycle policy minimum duration gotchas:
├── Standard-IA:          30-day minimum storage charge
│   Delete on day 29: charged for 30 days anyway
├── Glacier Instant:      90-day minimum
├── Glacier Flexible:     90-day minimum
└── Glacier Deep Archive: 180-day minimum

Transition cost math:
Don't transition objects that are deleted before minimum duration!
If avg object lifetime < 90 days → Glacier Instant is more expensive

Example:
Log file created → kept 60 days → deleted
Transition to Standard-IA at day 30:
├── Storage: 30 days × $0.0125/GB ✓
└── Cost: paid for 30 days of IA ✓ (fine — object lives past 30 days)
```

```
Log file → Glacier at day 30 → deleted at day 60:
└── Cost: charged for 90-day minimum even though deleted at 60 days!
    → Glacier wrong for 60-day-lived objects
```

🧩 Storage Cost Optimization Patterns

```
Pattern 1: Data Lake cost optimization
Raw data in:   S3 Standard-IA (accessed occasionally for reprocessing)
Processed data: S3 Standard (frequently queried by Athena)
Archive:        S3 Glacier Deep Archive after 1 year

Pattern 2: Backup optimization
Application backups:
├── Latest 7 days:  Standard (rapid recovery for recent issues)
├── Days 7-30:      Standard-IA (monthly restore for partial recovery)
└── Days 30+:       Glacier Deep Archive (annual DR testing only)

Pattern 3: Large objects vs small objects
└── Large objects (>1MB): any tier transition works well
    Small objects (<128KB): avoid Intelligent-Tiering (monitoring fee >
savings)
    Small objects: use Standard if frequently accessed,
                   manually manage lifecycle if access is predictable

Pattern 4: S3 request cost optimization
S3 Standard: $0.0004/1,000 GET requests
100 million GETs/month = $40/month in GET costs alone (not just storage!)

Reduce GET requests:
├── CloudFront in front of S3: serve from edge cache → fewer S3 GETs
├── S3 Select: retrieve subset of object without downloading entire file
│   SELECT * FROM S3Object WHERE price > 100 → reduces data transferred
└── Batch operations: combine operations instead of individual GETs

Pattern 5: Multipart upload + abort lifecycle
# Large uploads use multipart upload
# Incomplete multipart uploads accumulate in S3 silently!
# Cost: charged for all uploaded parts, even incomplete uploads

# Add lifecycle rule to abort incomplete multipart uploads
aws s3api put-bucket-lifecycle-configuration \
  --bucket my-bucket \
  --lifecycle-configuration '{
    "Rules": [{
      "ID": "abort-incomplete-multipart",
      "Status": "Enabled",
      "Filter": {},
      "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7}
    }]
```

```
  }'
# Clean up after 7 days → prevents silent cost accumulation
```

💬 Short Crisp Interview Answer

"S3 has seven storage classes from Standard at $0.023/GB to Glacier Deep Archive at $0.001/GB. Intelligent-Tiering is the default choice for objects with unpredictable access — it automatically moves objects between frequent and infrequent tiers based on 30-day access windows, with no retrieval fees on access from any tier. The monitoring fee of $0.00025/object/month makes it uneconomical for objects under 128KB. For predictable patterns (logs, backups), use lifecycle policies to transition through Standard → Standard-IA → Glacier → Deep Archive on specific day schedules. Three common gotchas: minimum storage duration charges (30 days for Standard-IA, 90 days for Glacier, 180 days for Deep Archive), incomplete multipart uploads silently accumulating cost, and S3 GET request costs adding up at high request volumes."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Minimum storage duration	Deleting before minimum = charged for full minimum anyway
IT monitoring fee for small objects	Objects < 128KB: monitoring fee exceeds storage savings in Intelligent-Tiering
Incomplete multipart uploads	Parts accumulate silently — always add AbortIncompleteMultipartUpload lifecycle rule
One Zone-IA = single AZ	Data loss if that AZ is destroyed — don't use for unique, irreplaceable data
Glacier retrieval costs	Deep Archive: $0.02/GB retrieval. Restoring 10TB = $200 plus the restore wait
Expedited Glacier retrieval	Expedited = 1-5 min but costs $0.03/GB + $0.01/request — expensive surprise
Lifecycle transition PUT cost	Each lifecycle transition incurs a PUT request cost per object transitioned

---

11.6 Data Transfer Costs — The Hidden AWS Bill Killer

🟢 What It Is in Simple Terms

AWS charges for data moving OUT of AWS (egress) and between certain regions and services. These costs are almost always underestimated — a well-designed architecture that handles 10TB/day of traffic can have

$5,000-$50,000/month in hidden data transfer fees. Understanding exactly what costs what is essential for any architect.

🔍 The Data Transfer Cost Model

```
What is FREE (no charge):
├── Data IN to AWS from internet: always free (ingress)
├── Traffic within same AZ: free (e.g., EC2 → EC2 in same AZ)
├── Traffic within same region using private IPs: most services free
│   Exception: inter-AZ traffic within a region = NOT free
├── CloudFront → S3 origin (same region): free
├── S3 → CloudFront: free (no origin transfer charge)
└── AWS services → internet (some services): varies

What COSTS money:
├── Data OUT to internet: $0.09/GB first 10TB, declining tiers
├── Inter-region transfers: $0.02/GB (US-EU), varies by regions
├── Inter-AZ transfers: $0.01/GB each direction ($0.02/GB round trip!)
├── NAT Gateway: $0.045/GB processed (in ADDITION to EC2 instance cost)
├── PrivateLink/VPC Endpoints: $0.01/GB processed
└── VPN / Direct Connect: separate pricing models
```

🧩 Data Transfer Cost Categories

```
1. INTERNET EGRESS (most significant cost for public-facing services):
   First 10 TB/month: $0.09/GB
   Next 40 TB/month:  $0.085/GB
   Next 100 TB/month: $0.07/GB
   Over 150 TB/month: $0.05/GB

   Example: Video streaming service sends 100TB/month to users
   $0.09 × 10,000 GB + $0.085 × 40,000 GB = $900 + $3,400 = $4,300/month
   → Use CloudFront: $0.0085/GB at edge (10-40TB) = much cheaper

2. INTER-AZ TRAFFIC (most commonly overlooked):
   Cost: $0.01/GB per direction (bidirectional = $0.02/GB round trip)

   Hidden cost scenario:
   EC2 in us-east-1a → EC2 in us-east-1b: $0.01/GB
   EC2 in us-east-1b → EC2 in us-east-1a: $0.01/GB
   Total: $0.02/GB for each GB of request+response data

   Microservices inter-AZ example:
   Service A (AZ-a) → Service B (AZ-b): 500 GB/day of API traffic
   500 GB × 2 directions × $0.01/GB × 30 days = $300/month
   Scale to 5TB/day: $3,000/month just for inter-AZ service mesh traffic!

3. INTER-REGION TRAFFIC:
```

```
   US ↔ EU:     $0.02/GB
   US ↔ Asia:   $0.08/GB
   EU ↔ Asia:   $0.09/GB

   Common pattern: DynamoDB Global Tables, RDS replication, cross-region
backups
   Each GB replicated across regions = inter-region transfer cost

4. NAT GATEWAY (the silent bill killer):
   Cost: $0.045/GB processed (EACH GB flowing through NAT Gateway)
   PLUS: $0.045/hr for the NAT Gateway itself (~$32/month for a running
NAT GW)

   NAT Gateway cost example:
   50 EC2 instances in private subnets
   Each downloads software updates: 500MB/day each
   50 × 500MB = 25GB/day through NAT Gateway
   25GB × $0.045 × 30 days = $33.75/month just for updates

   Large data processing pipeline:
   Lambda functions in VPC downloading S3 data through NAT Gateway
   10TB/day × $0.045/GB × 30 days = $13,500/month!
   Fix: S3 Gateway Endpoint (free!) — Lambda → S3 without NAT Gateway
```

🧩 Inter-AZ Traffic Optimization

```
Problem: Multi-AZ architecture is required for HA.
         But inter-AZ traffic costs $0.01/GB per direction.
         High-traffic microservices can spend thousands/month.

Solutions:

1. AZ-aware routing (locality-based routing):
   Configure services to prefer calls within same AZ.
   ALB → Target Group → EC2: prefer same-AZ targets first.

   ECS/EKS: use Topology Aware Routing:
   # Kubernetes service with topology hints
   apiVersion: v1
   kind: Service
   metadata:
     annotations:
       service.kubernetes.io/topology-mode: "auto"
   # Kubernetes routes to endpoints in same AZ first
   # Falls back to other AZs only if local AZ has no healthy endpoints

2. Service mesh AZ awareness:
   AWS App Mesh / Istio: route requests to same-AZ instances
   using weighted routing based on zone metadata.
```

```
3. ELB cross-zone load balancing cost:
   NLB: $0.006/GB for cross-AZ traffic through NLB (if enabled)
   ALB: cross-zone LB enabled by default — incurs $0.008/GB cross-AZ
   → For intra-cluster traffic: consider disabling cross-zone LB
      if your service has uniform instance distribution across AZs

4. Measure inter-AZ traffic:
   VPC Flow Logs → Athena query:
   SELECT sourceAZ, destinationAZ, SUM(numBytes)
   FROM flow_logs
   WHERE sourceAZ != destinationAZ
   GROUP BY sourceAZ, destinationAZ
   ORDER BY SUM(numBytes) DESC
```

🧩 NAT Gateway Cost Optimization

```
NAT Gateway is one of the top 5 largest AWS bills for many companies.
Every GB of private subnet traffic to internet/AWS services flows through
it.

Architecture review:
Private Lambda/EC2 → NAT Gateway → Internet/AWS Service
     ↑                ↑
     No charge    $0.045/GB ← the problem

Fix 1: S3 and DynamoDB Gateway Endpoints (FREE):
# S3 Gateway Endpoint: traffic to S3 bypasses NAT Gateway entirely
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-123 \
  --service-name com.amazonaws.us-east-1.s3 \
  --route-table-ids rtb-private-1 rtb-private-2

# DynamoDB Gateway Endpoint: also free
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-123 \
  --service-name com.amazonaws.us-east-1.dynamodb \
  --route-table-ids rtb-private-1 rtb-private-2

After: Lambda → S3 via Gateway Endpoint → $0 (no NAT Gateway charge)
Before: Lambda → NAT Gateway → S3 → $0.045/GB
Savings: 100% of S3/DynamoDB traffic cost eliminated

Fix 2: Interface Endpoints for other AWS services:
# Interface endpoints cost: $0.01/GB (vs $0.045/GB for NAT Gateway)
# For: SSM, ECR, CloudWatch, KMS, Secrets Manager, STS
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-123 \
  --vpc-endpoint-type Interface \
  --service-name com.amazonaws.us-east-1.ssm
```

```
Cost comparison:
Lambda pulls 10TB/month from ECR in VPC:
Via NAT Gateway:      10,000 GB × $0.045 = $450/month
Via Interface Endpoint: 10,000 GB × $0.01 = $100/month + $0.01/hr endpoint
Savings: $350/month

Fix 3: Lambda outside VPC (if no VPC resources needed):
Lambda without VPC: connects directly to AWS services (free)
Lambda with VPC: must go through NAT Gateway for internet + some AWS
services
→ Only put Lambda in VPC if it genuinely needs to access VPC resources
   (RDS, ElastiCache, EC2 on private subnet)
→ Lambda that only calls S3/DynamoDB/SQS doesn't need VPC

Fix 4: NAT Gateway per AZ (required for HA):
# Route tables: private subnets in AZ-a → NAT GW in AZ-a
# Route tables: private subnets in AZ-b → NAT GW in AZ-b
# This avoids cross-AZ traffic to reach NAT Gateway!
# Without this: AZ-b instances → AZ-a NAT GW = $0.01/GB cross-AZ PLUS
$0.045/GB NAT = $0.055/GB
```

🧩 CloudFront as Data Transfer Optimizer

```
Internet egress from EC2/ALB: $0.09/GB (us-east-1)
CloudFront egress to users:   $0.0085/GB (first 10TB, US)

Savings: $0.09 vs $0.0085 = 90% reduction on egress costs
PLUS: cache hit rate (60-90% for typical content) = even more reduction

CloudFront cost example:
Web app serves 100TB/month to US users

Without CloudFront:
100,000 GB × $0.085/GB avg = $8,500/month

With CloudFront (80% cache hit rate):
├── 20% origin requests: 20,000 GB from ALB → CloudFront = free
│   (ALB to CloudFront same region = free)
└── 100,000 GB × $0.0085/GB = $850/month from CF to users
Total: $850/month (vs $8,500/month) → 90% savings

CloudFront price classes:
├── All Edge Locations: global, highest cost (but lowest per-GB)
├── 200: all locations except most expensive
└── 100: North America + Europe only (lower cost, higher latency
elsewhere)

S3 Transfer Acceleration:
For uploads FROM users TO S3 globally:
Cost: $0.04/GB (US/EU) on top of S3 standard rates
```

```
  Benefit: 50-500% faster uploads using CloudFront edge → AWS backbone
  Use for: large file uploads where upload speed matters to user experience
```

🧩 Cost Analysis Queries

```
  AWS Cost Explorer: find your data transfer costs
  aws ce get-cost-and-usage \
    --time-period Start=2024-01-01,End=2024-02-01 \
    --granularity MONTHLY \
    --filter '{"Dimensions":{"Key":"SERVICE","Values":["Amazon Elastic
  Compute Cloud - Compute"]}}' \
    --group-by '[{"Type":"DIMENSION","Key":"USAGE_TYPE"}]' \
    --metrics BlendedCost

  # Filter for transfer usage types:
  # DataTransfer-Out-Bytes:    internet egress from EC2
  # DataTransfer-Regional-Bytes: inter-region and inter-AZ
  # NatGateway-Bytes:          NAT Gateway data processed

  VPC Flow Logs → Athena: find top talkers
  SELECT
    sourceAddress,
    destinationAddress,
    SUM(numBytes)/1024/1024/1024 AS totalGB,
    COUNT(*) AS flowCount
  FROM vpc_flow_logs
  WHERE action = 'ACCEPT'
    AND year = '2024' AND month = '01'
  GROUP BY sourceAddress, destinationAddress
  ORDER BY totalGB DESC
  LIMIT 20;

  # Identify: which EC2 → which destination generates most traffic?
  # Common finding: Lambda in VPC downloading from S3 through NAT GW
  # Fix: S3 Gateway Endpoint → $0 data transfer cost
```

💬 Short Crisp Interview Answer

"Data transfer is the most consistently underestimated AWS cost. The key categories: internet egress at $0.09/GB (fix with CloudFront at $0.0085/GB for 90% savings), inter-AZ traffic at $0.01/GB per direction (fix with topology-aware routing to prefer same-AZ targets), and NAT Gateway at $0.045/GB processed (the silent killer). For NAT Gateway, the first fix is always: add S3 and DynamoDB Gateway Endpoints — they're free and eliminate all S3/DynamoDB traffic from NAT Gateway. Interface Endpoints for SSM, ECR, and CloudWatch cost $0.01/GB vs $0.045/GB through NAT Gateway. Never put Lambda in a VPC unless it genuinely needs to access VPC resources. Use VPC Flow Logs + Athena to identify your top traffic flows and trace them to specific source/destination pairs."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Inter-AZ is bidirectional	Request + response both charged: 1GB request/response = $0.02/GB total
NAT Gateway per AZ = required	One NAT GW in one AZ → all other AZ traffic crosses AZs = $0.055/GB (NAT + inter-AZ)
S3 Gateway Endpoint is FREE	Many teams pay for S3 NAT Gateway traffic unnecessarily — Gateway Endpoints cost nothing
Lambda in VPC = NAT required	Lambda in VPC needs NAT Gateway for any non-VPC endpoint AWS service calls
CloudFront to S3 same region = free	S3 doesn't charge for data transferred to CloudFront in the same region
Data Transfer Acceleration cost	S3 Transfer Acceleration adds $0.04/GB on top of storage costs — only when speed matters
Cross-region replication cost	DynamoDB Global Tables and RDS cross-region replication incur full inter-region transfer fees

---

🔗 Category 11 — Full Connections Map

```
COST OPTIMIZATION connects to:

Cost Explorer / Budgets / Anomaly Detection
├── Tags                → cost allocation requires activated tags in
Billing
├── Organizations       → consolidated billing view across all accounts
├── CloudWatch          → Budget alarm → SNS → PagerDuty
├── SNS                 → budget and anomaly alerts delivered via SNS
├── Lambda              → Budget Actions trigger Lambda for auto-
remediation
└── Cost and Usage Report → detailed billing data for custom analysis

Savings Plans / Reserved Instances
├── EC2                 → compute SP and EC2 SP reduce instance costs
├── Lambda              → Compute SP covers Lambda $/GB-second
├── Fargate             → Compute SP covers Fargate vCPU + memory
├── RDS                 → RDS Reserved Instances (SPs don't cover RDS)
└── Cost Explorer       → RI/SP utilization and coverage reports

Trusted Advisor / Compute Optimizer
├── CloudWatch          → Compute Optimizer reads CW metrics for analysis
├── CloudWatch Agent    → enables memory metrics for better
recommendations
```

```
  ├── Cost Explorer       → rightsizing recommendations also in Cost
Explorer
  ├── EventBridge         → TA check status changes trigger automation
  └── Support API         → programmatic access to all TA checks

  Spot Instances
  ├── Auto Scaling Groups → mixed instances policy with Spot + On-Demand
  ├── EventBridge         → EC2 Spot Instance Interruption Warning events
  ├── Lambda              → interruption handler auto-remediates
  ├── ELB                 → deregister Spot instances before interruption
  └── EMR / Batch         → native Spot support for task nodes

  S3 Storage Tiers
  ├── S3 Lifecycle        → automated tier transitions over time
  ├── S3 Intelligent-Tiering → automatic tier selection by access pattern
  ├── CloudFront          → reduce S3 GET costs with edge caching
  ├── Athena              → Parquet in S3 reduces scan cost significantly
  ├── Glue                → schema registry for Parquet conversion
  └── S3 Analytics        → analyze access patterns before choosing tier

  Data Transfer
  ├── CloudFront          → reduce internet egress 90%
  ├── NAT Gateway         → $0.045/GB — largest hidden cost
  ├── VPC Endpoints       → S3/DynamoDB Gateway Endpoints (free)
  ├── Interface Endpoints → $0.01/GB for SSM, ECR, CloudWatch (vs $0.045
NAT)
  ├── VPC Flow Logs       → identify top traffic flows for optimization
  └── Athena              → query flow logs to find expensive traffic
patterns
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Fact
Cost Explorer data delay	8-24 hour delay — yesterday's costs may not appear until today
Cost Explorer tag filter	Must activate cost allocation tags in Billing console before CE can filter by them
Budget notification types	ACTUAL (has happened) and FORECASTED (will happen) — use both
Budget Actions	Can automatically stop EC2, apply IAM policy, or target SCP when threshold hit
Anomaly Detection training	Needs ~10 days of data before useful detection. No thresholds to set
Anomaly root cause	Alert includes which service, region, usage type, account caused the spike
Compute SP flexibility	ANY EC2 family/size/region + Lambda + Fargate — most flexible committed discount

Topic	Key Fact
EC2 Instance SP	Specific instance family in specific region — higher discount, less flexible
Savings Plans vs RDS	Savings Plans do NOT cover RDS — RDS needs its own Reserved Instances
Zonal RI capacity	Only Zonal RI reserves physical EC2 capacity — Regional RI does not
Commitment sizing	Commit to 70-80% of MINIMUM usage — not peak. Over-commitment wastes money
RI/SP utilization target	> 90% utilization (if lower: over-bought). > 80% coverage (if lower: opportunity)
Trusted Advisor tiers	Basic: ~6 checks. Business/Enterprise: 200+ checks across all 5 pillars
Compute Optimizer memory	Needs CloudWatch Agent on EC2 for memory metrics — without it, recommendations incomplete
Compute Optimizer lookback	14 days default. 3-month option available for better seasonal analysis
Spot discount range	60-90% off On-Demand depending on instance type and AZ
Spot interruption notice	2-minute warning via IMDS metadata and EventBridge
Spot allocation strategy	price-capacity-optimized: best mix of price + available capacity
Spot minimum instance types	Use 6-8+ instance types across 3 AZs — never single type
OnDemandBaseCapacity	Always set 2-3 OD instances as guaranteed floor in mixed ASG
Spot for databases	NEVER use Spot for stateful databases — interruption risk is too high
S3 Standard cost	$0.023/GB/month. No minimum duration, no retrieval fee
S3 Intelligent-Tiering	No retrieval fees. Best for unknown access patterns. $0.00025/object/month monitoring fee
S3-IT monitoring fee threshold	Objects < 128KB: monitoring fee exceeds storage savings
S3 Standard-IA minimum	30-day minimum storage — delete before 30 days = charged for full 30 days
S3 Glacier Flexible minimum	90-day minimum storage
S3 Deep Archive minimum	180-day minimum. 12-48 hour retrieval time
S3 incomplete multipart uploads	Parts accumulate cost silently — add AbortIncompleteMultipartUpload lifecycle rule
S3 GET request cost	$0.0004/1,000 GETs — high-volume access needs CloudFront cache

Topic	Key Fact
Internet egress cost	$0.09/GB first 10TB. Reduce 90% with CloudFront at $0.0085/GB
Inter-AZ cost	$0.01/GB per direction — $0.02/GB round trip. Fix with topology-aware routing
NAT Gateway data cost	$0.045/GB processed — the silent bill killer
S3 Gateway Endpoint	FREE — eliminates all S3 + DynamoDB traffic from NAT Gateway
Interface Endpoint cost	$0.01/GB (vs $0.045/GB NAT) for SSM, ECR, CloudWatch, Secrets Manager
Lambda in VPC cost	Lambda in VPC needs NAT Gateway for AWS service calls (unless using endpoints)
NAT Gateway per AZ	Deploy one NAT Gateway per AZ — avoid cross-AZ traffic to reach NAT ($0.055/GB)
CloudFront to S3 same region	Free — S3 doesn't charge transfer to CloudFront in same region

Category 11: Cost Optimization — Complete Interview Guide
Excluded topics: CodeBuild, CodePipeline, CloudFormation